

Aim- Design a distributed application using Hadoop MapReduce for:

1. Character counting in a given text file
2. Counting number of occurrences of every word in a given text file

Introduction:

Hadoop MapReduce is a powerful programming model designed for processing and analyzing large volumes of data in a distributed computing environment. It is a core component of the Apache Hadoop ecosystem, enabling parallel processing across multiple nodes in a cluster. This model allows applications to scale efficiently by dividing large datasets into smaller chunks and processing them simultaneously.

The MapReduce model primarily consists of two phases: the Map phase and the Reduce phase, with an intermediate step called Shuffle and Sort.

1. Map Phase

Hadoop Map Reduce is a distributed programming model used for processing vast amounts of structured and unstructured data across clusters of computers. It is a core component of Apache Hadoop, designed to handle big data efficiently using parallel processing and fault tolerance mechanisms. The fundamental idea behind Map Reduce is to divide a large task into smaller sub-tasks, process them independently in parallel, and then combine the results to obtain the final output. The Map phase is the first step in the data processing pipeline. In this phase, the input data stored in the Hadoop Distributed File System (HDFS) is divided into smaller blocks, and each block is processed by a separate Mapper.

- The Mapper reads input data line by line.
- It processes the data and transforms it into intermediate key-value pairs.
- These key-value pairs serve as the basic units of data for further processing.

For example, in a word count application, the Mapper reads a sentence and emits pairs like:

(word, 1)

If the input is: “Hello Hadoop Hello”, the Mapper output will be:

(Hello, 1), (Hadoop, 1), (Hello, 1)

The Map phase is highly parallel, meaning multiple Mappers can run simultaneously on different data blocks, significantly improving performance.

2. Shuffle and Sort Phase

After the Map phase, the framework automatically performs the Shuffle and Sort operation.

- Shuffling: Groups all intermediate values associated with the same key.
- Sorting: Arranges keys in a sorted order before sending them to Reducers.

For example:

(Hello, 1), (Hello, 1) → (Hello, [1,1])
(Hadoop, 1) → (Hadoop, [1])

This phase ensures that all values corresponding to a particular key are brought together.

3. Reduce Phase

The Reduce phase processes the grouped data generated from the Shuffle phase.

- Each Reducer receives a key and a list of associated values.
- It performs aggregation or computation on these values.
- Finally, it produces the output in the form of key-value pairs.

Word count example:

(Hello, [1, 1]) → (Hello, 2)
(Hadoop, [1]) → (Hadoop, 1)

The output is then stored back in HDFS.

Architecture Overview:

The MapReduce framework follows Master-Slave architecture:

- JobTracker (Master):
 - Manages job scheduling
 - Assigns tasks to nodes
 - Monitors execution
- TaskTracker (Slave):
 1. Executes Mapper and Reducer tasks
 2. Reports progress to JobTracker

Advantages of MapReduce

- Scalability: Can process petabytes of data by adding more nodes.
- Fault Tolerance: Automatically handles node failures.

- Parallel Processing: Speeds up computation by distributing tasks.
- Data Locality: Moves computation closer to data rather than moving data.

Limitations of Map Reduce

- Not suitable for real-time processing
- High latency due to disk-based operations
- Complex for iterative algorithms
- Slower compared to modern frameworks like Apache Spark

Conclusion

Successfully implemented distributed applications using Hadoop MapReduce to perform character counting and word counting.